

VIISIRI Green Coding Guide

VIISIRI Green Coding Guide

Energy-Aware Software Development Across Domains

This guide presents a practical, evidence-based approach to reducing the **energy footprint of software systems** across frontend, backend, and AI domains.

Sections

1. [Introduction]({{ "/introduction/" | relative_url }})
2. [Defining Green Coding]({{ "/defining-green-coding/" | relative_url }})
3. [Core Principles]({{ "/core-principles/" | relative_url }})
4. [Practices by Domain]({{ "/domain-practices/" | relative_url }})
5. [Measurement & Feedback]({{ "/measurement-feedback/" | relative_url }})
6. [Workflow Integration]({{ "/workflows/" | relative_url }})
7. [Trade-offs & Limitations]({{ "/tradeoffs/" | relative_url }})
8. [Resource Footprint of Software]({{ "/footprint/" | relative_url }})
9. [Sustainability Handprint of Software]({{ "/handprint/" | relative_url }})
10. [Checklists]({{ "/checklists/" | relative_url }})
11. [Outlook]({{ "/outlook/" | relative_url }})

1. Introduction

Green Coding as a Software Quality Attribute

The energy footprint of software systems has become an increasingly important concern as digital infrastructure continues to scale. While early sustainability efforts in information and communication technology (ICT) focused primarily on hardware efficiency and data-center optimization, it is now widely acknowledged that **software itself plays a decisive role in determining runtime energy consumption**.

Software design and implementation choices directly influence how intensively hardware resources such as CPU, memory, network interfaces, storage devices, and accelerators are utilized. These usage patterns, in turn, determine the **energy footprint of software during execution**. Even small inefficiencies at the code or design level can accumulate into substantial energy consumption when software is executed at scale.

Modern software systems often operate continuously, serve large user populations, and rely on distributed architectures. In such contexts, green coding is not an isolated optimization task but a **software quality concern** that interacts with performance, reliability, scalability, and maintainability.

This guide approaches green coding as an engineering discipline grounded in **evidence and measurement**, rather than intuition or isolated best practices. It is written for a mixed audience of industry practitioners and researchers and aims to bridge research insights with day-to-day software development practice.

2. Defining Green Coding

2.1 What Is Green Coding?

The term *green coding* is used in both academic and industrial contexts to describe software development practices aimed at reducing the environmental impact of software systems. However, the literature reveals that there is **no single, universally accepted definition** of green coding. Instead, the concept appears under related terms such as *green software*, *energy-efficient software*, and *sustainable software engineering* (Hilty and Aebischer, 2015; Procaccianti et al., 2014).

Several representative definitions illustrate the range of perspectives:

- *Green and sustainable software* is often defined as software that “minimizes negative environmental impacts throughout its lifecycle” (Penzenstadler et al., 2014).
- *Energy-efficient software* is commonly described as software that “reduces energy consumption during execution while maintaining acceptable performance” (Procaccianti et al., 2014).
- *Sustainable software engineering* has been framed as the discipline of designing software systems with explicit consideration of long-term environmental, social, and economic impacts (Becker et al., 2016).

These definitions differ primarily in **scope**. Some adopt a broad lifecycle perspective, encompassing development, deployment, use, and disposal, while others focus more narrowly on **runtime behavior**. For the purposes of this guide, whose primary audience includes software developers and architects making concrete design and implementation decisions, a runtime-focused definition provides the greatest practical and empirical value.

Accordingly, this guide adopts the following working definition:

Green coding is the design and implementation of software in a way that avoids unnecessary runtime energy and resource consumption while meeting functional and quality requirements.

This definition highlights several important characteristics of green coding:

- The software energy footprint is primarily a **runtime property**, emerging during execution rather than during development alone.
- Energy consumption is **workload-dependent**, varying with input data, usage patterns, and deployment context.
- The energy footprint is shaped by **software decisions**, including algorithms, data structures, architectures, interaction patterns, and configuration choices.
- Energy-related effects are **observable through measurement or estimation**, enabling empirical evaluation rather than reliance on intuition.

By focusing on runtime energy consumption, this definition aligns green coding with established software quality concerns such as performance and scalability, while remaining compatible with broader sustainability perspectives that consider system-level and societal impacts. Green coding therefore applies across multiple levels of abstraction, from individual functions and methods to entire software architectures.

2.2 What Green Coding Is *Not*

Clarifying what green coding is *not* is essential to avoid common misconceptions that can limit its effectiveness or lead to misguided optimization efforts.

First, green coding is **not synonymous with performance optimization**. Although execution time and energy consumption are often correlated, they are not equivalent. Faster execution can increase power draw due to higher utilization of CPUs or accelerators, resulting in higher total energy consumption in some cases (Hackenberg et al., 2015). Conversely, slightly slower execution may reduce energy usage by allowing hardware components to operate in more energy-efficient states. Performance metrics alone are therefore insufficient as proxies for energy efficiency.

Second, green coding is **not a one-time activity**. Software systems evolve continuously through feature additions, refactoring, dependency updates, and configuration changes. Each of these changes can introduce energy regressions even when functional behavior remains unchanged. From this perspective, green coding is an **ongoing concern**, comparable to performance tuning or security hardening, rather than a single optimization phase.

Third, green coding is **not about premature micro-optimization**. Isolated low-level optimizations performed without empirical validation often increase code complexity without delivering meaningful reductions in energy footprint. Empirical studies in sustainable software engineering emphasize that the most significant energy savings typically arise from high-level design and architectural decisions, validated through measurement (Procaccianti et al., 2016).

Finally, green coding does not imply sacrificing correctness, maintainability, or other quality attributes by default. Instead, it promotes **explicit and informed trade-offs**, where energy efficiency is considered alongside functional and non-functional requirements. This perspective positions green coding as an integral part of responsible software engineering practice rather than an optional or secondary concern.

References

Becker, C., Betz, S., Chitchyan, R., Duboc, L., Easterbrook, S.M., Penzenstadler, B., Seyff, N. and Venters, C.C. (2016) *Requirements: The key to sustainability*. IEEE Software, 33(1), pp.56–65.

Hackenberg, D., Schöne, R., Ilsche, T., Molka, D., Schuchart, J. and Geyer, R. (2015) *An energy efficiency feature survey of the Intel Haswell processor*. IEEE International Parallel and Distributed Processing Symposium Workshops, pp.896–904.

Hilty, L.M. and Aebischer, B. (2015) *ICT for sustainability: An emerging research field*. In: Hilty, L.M. and Aebischer, B. (eds.) *ICT Innovations for Sustainability*. Springer, pp.3–36.

Penzenstadler, B., Bauer, V., Calero, C. and Franch, X. (2014) *Sustainability in software engineering: A systematic literature review*. Proceedings of the 16th International Conference on Evaluation & Assessment in Software Engineering (EASE), pp.1–10.

Procaccianti, G., Lago, P. and Vetro', A. (2014) *Energy efficiency in software: A systematic literature review*. Journal of Systems and Software, 97, pp.135–155.

Procaccianti, G., Lago, P. and Lewis, G. (2016) *Green architectural tactics for the cloud*. Proceedings of the 2016 IEEE/ACM International Conference on Software Architecture (ICSA), pp.41–50.

3. Core Principles of Green Coding

The following principles provide a technology-agnostic foundation for reducing the runtime energy footprint of software systems. They apply across domains and abstraction levels, from individual functions to distributed architectures. Domain-specific practices that implement these principles are described in [Section 4](#).

3.1 Avoid Unnecessary Work

The most effective way to reduce energy consumption is to avoid computation, data processing, and communication that provide no functional value. This includes logic that executes on every request but only matters in rare cases, data that is fetched but never used, and operations triggered by events that do not require them.

Unnecessary work is often invisible because it is spread across many small operations, each of which appears harmless in isolation. The cumulative effect across many users, requests, or devices can be substantial. Identifying it requires profiling rather than inspection, because the most expensive code is rarely the most obvious.

This principle also applies at the architectural level. Processing that could be avoided through better system design, such as repeatedly re-deriving the same result from a database, represents unnecessary work regardless of how efficiently that work is performed.

3.2 Avoid Repeated Work

Repeated execution of the same work increases cumulative energy footprint without adding functional value. Reuse, caching, and memoisation can significantly reduce redundant resource usage when applied deliberately and measured carefully. Empirical evaluation of API design variants has shown that introducing in-memory caching for repeated queries can reduce power draw by approximately 15%, while replacing inefficient algorithmic logic with optimised equivalents can yield reductions of up to 23% (Joof et al., 2025; Joof, 2025).

The key qualifier is *deliberately and measured*. Caching is not universally beneficial: it consumes memory, introduces staleness risk, and only saves energy when the hit rate is high enough to justify the overhead. The same applies to precomputation, which shifts work from request time to build or startup time. This may reduce per-request energy but increases system-level energy if the precomputed results are rarely used.

Repeated work is a particularly common problem in distributed systems, where the same data is fetched independently by multiple components, and in frontend applications, where components re-render in response to state changes that do not affect them.

3.3 Move and Store Less Data

Data movement and storage contribute significantly to energy consumption,

particularly in distributed systems and mobile environments. Transmitting data over a network, reading and writing to disk, and copying data between memory regions all consume energy in proportion to the volume transferred.

Reducing payload sizes, eliminating redundant serialisation and deserialisation, compressing data where the CPU cost of compression is outweighed by the energy savings from reduced transfer, and avoiding unnecessary persistence are all practical expressions of this principle.

The principle applies at every scale: reducing the number of columns selected in a database query, trimming the fields returned by an API, filtering log output, and removing unused dependencies from a software bundle are all instances of moving and storing less data.

3.4 Prefer Efficient Abstractions

Abstractions improve maintainability but can hide expensive operations. A convenient method call, a high-level framework feature, or a widely-used library may conceal significant computational overhead, particularly when used in a hot path, at scale, or in a resource-constrained environment.

Green coding encourages awareness of the resource and energy implications of the abstractions in use. This does not mean abandoning abstractions in favour of low-level code, because in most cases the maintainability cost of doing so far outweighs the energy benefit. It means being deliberate: understanding what an abstraction does under the hood, choosing lighter alternatives when they exist and are functionally equivalent, and profiling rather than assuming that a familiar tool is efficient.

This principle is particularly relevant when selecting frameworks, runtimes, serialisation formats, and communication protocols, where the choice affects the entire system rather than a single operation.

3.5 Make Energy Footprint Visible

Without measurement, energy efficiency remains speculative. Developers cannot reliably identify the most energy-intensive parts of a system through inspection alone, and energy improvements cannot be verified without a baseline to compare against.

Visibility through profiling and comparison is essential for evidence-based green coding. This means establishing energy or resource baselines before optimising, measuring the impact of changes rather than assuming them, and making energy-related information available to the team through dashboards, CI metrics, or code review artefacts, so that it can inform decisions over time.

Visibility also means acknowledging uncertainty. Energy measurement tools have limitations (see [Section 7](#)), and results should be interpreted comparatively and transparently rather than treated as precise absolute values. Prototype research integrating energy profiling directly into version control workflows has shown that commit-level energy feedback increases developer awareness of sustainability and influences design decisions, even among developers who had not previously considered energy as a quality concern (Joof et al., 2025; Joof, 2025).

References

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta–Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyediji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.

4. Green Coding Practices by Domain

Green coding principles apply across domains, but their practical impact depends on execution context. This section organises practices by domain, focusing on **dominant contributors to runtime energy footprint**. The practices below expand on the core principles from [Section 3](#) and are intended to give practitioners a concrete starting point for each domain.

4.1 Frontend – Web Applications

Energy footprint drivers include JavaScript execution, rendering, and network activity. Because the same code runs on millions of user devices, frontend inefficiency is multiplied across the entire user base, making even small per-page improvements significant at scale (Manotas et al., 2016).

Minimise unnecessary re-renders and layout thrashing In component-based frameworks, a state change in a parent can trigger re-renders in subtrees that are unaffected by the change. Each re-render involves diffing, DOM patching, style recalculation, and potentially layout, all of which is CPU work that generates heat on the user's device. Use memoisation, selector functions, and component splitting to ensure that only the components that need to update actually do. Similarly, avoid interleaving DOM reads and writes in JavaScript loops, which forces the browser to repeatedly recalculate layout (a pattern known as layout thrashing).

Reduce bundle size through code splitting and lazy loading Larger JavaScript bundles require more parsing, compilation, and execution time, all of which consume energy before a single line of application logic runs. Code splitting ensures that users download and execute only the code needed for their current view. Lazy loading defers the rest until it is actually required. Audit bundle composition regularly using build analysis tools, and remove unused dependencies.

Avoid chatty API interactions Multiple small, sequential API calls keep the network interface active and increase latency. Each network request also has a fixed energy overhead independent of payload size: initiating a connection, performing DNS resolution, and completing the TCP/TLS handshake. Consolidate related requests, use HTTP/2 multiplexing where available, and consider query languages (such as GraphQL) that allow the client to specify exactly the data it needs.

Use caching and HTTP mechanisms effectively Redundant data transfer is one of the most avoidable sources of frontend energy consumption. Correct use of Cache-Control headers, ETags, and service workers can eliminate repeat fetches of unchanged resources entirely. Measure actual cache hit rates to confirm that caching is working as intended.

Prefer CSS and hardware-accelerated animations over JavaScript loops JavaScript-driven animations run on the main thread and compete with layout and rendering. CSS transitions and the Web Animations API delegate work to the compositor thread, which is typically GPU-accelerated and does not block main thread execution. Use transform and opacity for animations, as these properties can be composited without triggering layout or paint.

4.2 Frontend – Mobile Applications

Mobile energy footprint is dominated by battery usage from the CPU, network radios, display, and sensors. The cost of waking the device from a low-power state is significant: a single unnecessary wakeup can consume as much energy as many milliseconds of normal CPU activity (Pathak et al., 2012).

Reduce background activity and wakeups Each background wakeup forces the CPU and potentially the network radio out of a low-power sleep state. Consolidate background tasks using platform-provided APIs such as WorkManager on Android and BGTaskScheduler on iOS, which allow the OS to schedule work during periods when the device is already active or charging. Avoid independent timers and alarms for tasks that can tolerate batching.

Batch network requests and support offline workflows The cellular radio (LTE/5G) is one of the most energy-intensive components in a mobile device and consumes significant power even in the tail period after a transmission ends, while waiting for further activity. Batching requests into fewer, larger interactions reduces the total time the radio spends in its active and tail states. Supporting offline workflows, by storing and replaying operations when connectivity is restored, further reduces unnecessary transmission.

Optimise rendering and scrolling Off-screen rendering, excessive overdraw (drawing the same pixel multiple times per frame), and repeated layout passes drain battery directly. Use view recycling (e.g., RecyclerView, LazyColumn) to avoid rendering off-screen content. Reduce overdraw by flattening view hierarchies and removing opaque backgrounds that are hidden by foreground content.

Use sensors conservatively and at appropriate precision GPS at full accuracy and high polling rate is among the most energy-intensive sensors on a mobile device. Where coarse location is sufficient, use `PRIORITY_LOW_POWER` or geofencing rather than continuous high-accuracy tracking. Apply the same principle to accelerometers, gyroscopes, and microphones: request the lowest sampling rate and precision that satisfies the functional requirement, and unregister listeners when they are no longer needed.

4.3 Backend – Services and APIs

Backend energy footprint is driven by CPU usage per request, data access patterns, and network communication volume. Because a single backend instance serves many concurrent users, inefficiencies compound rapidly with traffic growth.

Reduce work per request Profile hot paths to identify operations that execute on every request. Eliminating or deferring even a single expensive database call, computation, or external service call from a high-traffic endpoint can yield significant aggregate energy savings across the server fleet. Treat per-request CPU time as a resource budget that should be defended as the codebase evolves. In a controlled study comparing API design variants, replacing an $O(n^2)$ nested-loop sort with an optimised built-in equivalent reduced power draw by approximately 23% and response time by 43% relative to the unoptimised baseline (Joof et al., 2025; Joof, 2025).

Avoid N+1 queries and over-fetching Loading a list of entities followed by individual queries for each entity is one of the most common and avoidable energy waste patterns in backend development. Replace N+1 patterns with a single batch query or an eager-loading mechanism provided by the ORM or query builder. Equally important is selecting only the columns required, because fetching entire rows when only two fields are needed wastes I/O bandwidth and increases serialisation overhead.

Use bounded, measured caching Caching is only beneficial if the hit rate justifies the memory footprint and staleness risk. An unbounded cache increases GC pressure and memory consumption; a cache with the wrong eviction policy may hold stale or rarely accessed entries. Instrument caches to measure hit rates, and set TTLs and size limits based on observed access patterns rather than defaults. Empirical results from API-level profiling show that introducing in-memory caching for repeated queries reduces power consumption by approximately 15% and response time by 36% relative to an uncached baseline (Joof et al., 2025; Joof, 2025).

Minimise payload size and serialisation overhead JSON serialisation and deserialisation is CPU-intensive at scale. For internal service-to-service communication, binary formats such as Protocol Buffers or MessagePack can reduce both CPU time and transfer volume. For external-facing APIs, support response field filtering (e.g., via fields query parameters or GraphQL selections) so that clients are not forced to receive and discard data they do not use. Note that payload compression involves an explicit trade-off: GZIP encoding reduces network transfer but increases CPU consumption, and the net energy impact depends on the relative cost of transmission versus processing in the deployment environment (Joof et al., 2025; Joof, 2025).

4.4 Backend – Data Processing and Storage

Data pipelines consume energy through prolonged execution and large-scale I/O. Unlike interactive services, pipelines often run without a user waiting for a response, which means their energy consumption is less visible but no less real.

Avoid full recomputation of unchanged data Re-running a pipeline over input data that has not changed since the last run is pure waste. Detect unchanged partitions using checksums, modification timestamps, or incremental computation frameworks (such as Apache Spark's structured streaming or dbt's incremental models), and skip them. Even a partial reduction in recomputation scope can substantially reduce total pipeline energy.

Use incremental and checkpointed processing Long-running batch jobs that restart from scratch on failure waste all the energy invested in the failed run. Checkpointing, which periodically saves intermediate state to durable storage, enables the pipeline to resume from the last stable point rather than the beginning. The energy cost of writing checkpoints is typically small relative to the cost of a full restart.

Apply data retention and lifecycle policies Storage energy grows with data volume: more data means more disk reads, more index maintenance, and more backup and replication overhead. Define and enforce retention policies that expire data when it is no longer needed for operational, analytical, or compliance purposes. Move cold data, such as infrequently queried historical records, to lower-power storage tiers rather than keeping it on high-performance primary storage.

Align processing frequency with actual needs Running an hourly pipeline to produce results that are consumed daily is unnecessary work. Review the scheduling frequency of each pipeline in relation to the actual decision latency requirement of its downstream consumers. Where a delay is acceptable, prefer event-driven triggers over fixed schedules to avoid processing empty or minimal

increments.

4.5 Backend – Infrastructure and Cloud

Infrastructure choices determine the baseline energy cost of running software before any request is served. Over-provisioned or poorly configured infrastructure wastes energy continuously, not just under load.

Right-size compute resources An instance running at 5% average CPU utilisation is consuming roughly the same idle power as one running at 60%, while delivering far less useful work per watt. Profile actual resource utilisation across representative workload periods and right-size instances accordingly. Cloud environments make this reversible, so start conservatively and scale based on observed demand rather than theoretical peaks.

Use scale-to-zero and serverless patterns for bursty workloads Services that are idle for significant portions of the day benefit from architectures that allow compute resources to scale down to zero when not in use. Serverless functions, container-based autoscaling (e.g., Knative, AWS Lambda, Google Cloud Run), and scheduled shutdown of non-production environments all reduce energy consumption during periods of low demand.

Consider the carbon intensity of deployment regions The same workload produces different lifecycle carbon emissions depending on where it runs. Cloud regions powered predominantly by renewable energy sources have substantially lower carbon intensity than those relying on coal or gas. For workloads where latency requirements allow geographic flexibility, such as batch jobs, asynchronous processing, or data pipelines, prefer regions with lower carbon intensity. Most major cloud providers publish real-time or historical carbon intensity data for their regions.

Enforce autoscaling and avoid sustained idle over-provisioning Fixed-capacity deployments that are sized for peak load remain fully provisioned and consuming energy during off-peak hours. Configure autoscaling policies that reduce replica count, instance size, or cluster node count during sustained low-traffic periods. Combine this with scheduled scaling for predictable traffic patterns (e.g., scaling down overnight for services with a known diurnal usage pattern).

4.6 AI Development – Training

AI training energy footprint is dominated by accelerator (GPU/TPU) runtime. Training a large model can consume hundreds to thousands of kilowatt-hours, which often exceeds the lifetime inference energy for many deployments (Strubell et al., 2019; Patterson et al., 2021).

Right-size models and datasets Larger models and datasets do not always produce proportionally better results for a given task. Evaluate whether a smaller, fine-tuned, or distilled model meets quality requirements before committing to large-scale training. Similarly, audit training datasets for duplicates and low-quality samples that increase compute time without improving generalisation.

Avoid redundant experiments Training runs with near-identical hyperparameters, or re-running an experiment that was interrupted without saving a checkpoint, are common sources of avoidable energy consumption. Use experiment tracking tools (such as MLflow or Weights & Biases) to record configurations and results, and establish convergence criteria before starting a run rather than judging retroactively.

Apply early stopping and efficient training techniques Monitor validation metrics throughout training and stop when improvement plateaus rather than running for a fixed number of epochs. Mixed-precision training (FP16 or BF16) reduces memory bandwidth consumption and computation time per step on supported hardware, typically without material quality loss. Learning rate scheduling, gradient accumulation, and efficient attention implementations are further well-established techniques for reducing total training compute.

Maximise hardware utilisation Low GPU utilisation during training, commonly caused by data loading bottlenecks, preprocessing on the CPU, or inefficient batching, means energy is consumed without productive work. Profile GPU utilisation and address pipeline stalls by prefetching data, increasing the number of data loader workers, or preprocessing and caching inputs offline.

4.7 AI Usage – Inference

Inference energy footprint scales with request volume. Unlike training, which occurs once per model version, inference runs continuously in production and its

cumulative energy cost often exceeds the training cost over the deployment lifetime of a model.

Use the smallest effective model Model complexity should be matched to task requirements. A large general-purpose model applied to a narrow, well-defined task is likely to be significantly over-specified. Techniques including quantisation (reducing numerical precision), pruning (removing low-weight connections), and distillation (training a smaller model to replicate a larger one's behaviour) can substantially reduce inference energy without proportional loss in output quality.

Minimise prompt and input size In transformer-based models, self-attention complexity scales quadratically with input length. Longer prompts and larger context windows directly increase per-request compute cost. Trim irrelevant context, avoid verbose prompt templates, and evaluate whether retrieved context snippets can be shorter without reducing answer quality.

Cache deterministic outputs and embeddings Identical or semantically equivalent queries that reliably produce the same output should be served from cache rather than triggering re-inference. Embedding computations for fixed reference documents should similarly be precomputed and cached rather than recalculated on each request. Semantic caching, which matches queries that are paraphrases of each other, can extend cache hit rates beyond exact-match scenarios.

Batch requests where feasible Batching amortises the fixed cost of loading model weights from memory and allows the accelerator to operate at higher utilisation. For asynchronous or near-real-time use cases, accumulate requests into batches and process them together. Dynamic batching, which groups requests that arrive within a short time window, can achieve most of the throughput benefit without imposing significant additional latency.

References

Manotas, I., Bird, C., Zhang, R., Shepherd, D., Jaspan, C., Sadowski, C., Pollock, L. and Clause, J. (2016) *An empirical study of practitioners' perspectives on green software engineering*. Proceedings of the 38th International Conference on Software Engineering (ICSE), pp.237–248.

Pathak, A., Hu, Y.C. and Zhang, M. (2012) *Where is the energy spent inside my app? Fine-grained energy accounting on smartphones with Eprof*. Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys), pp.29–42.

Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L.M., Rothchild, D., So, D., Texier, M. and Dean, J. (2021) *Carbon emissions and large neural network training*. arXiv preprint arXiv:2104.10350.

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J.P. and Saraiva, J. (2017) *Energy efficiency across programming languages: How do energy, time, and memory relate?* Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering, pp.256–267.

Strubell, E., Ganesh, A. and McCallum, A. (2019) *Energy and policy considerations for deep learning in NLP*. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), pp.3645–3650.

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta–Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyediji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.

5. Measurement and Feedback

To be filled in by UTU

6. Integrating Green Coding into Development Workflows

Green coding must fit existing workflows to be sustainable as a practice. Principles and domain-specific knowledge are only useful if they reach developers at the

moment they make decisions: during implementation, review, and deployment, not as a retrospective audit. This section describes how energy awareness can be embedded into the stages of a typical development workflow.

Research into workflow-integrated energy profiling confirms that tools and processes which connect energy metrics to version control, continuous integration, and visual dashboards are both technically feasible and meaningfully adopted by developers, provided they require minimal setup effort and operate within tools developers already use (Joof et al., 2025).

6.1 Local Development

The cheapest place to identify an energy inefficiency is during local development, before code is reviewed, merged, or deployed. Profiling a change in isolation, comparing the energy or resource footprint of a new implementation against the previous version on representative test inputs, enables rapid experimentation with low overhead.

Lightweight profiling approaches that wrap existing tools such as PowerJoular in containerised, reproducible execution environments have demonstrated that per-request energy metrics (CPU usage, memory, power draw, response time) can be captured on accessible hardware such as a Raspberry Pi without requiring specialist instrumentation or privileged cloud access (Joof, 2025). This makes meaningful energy feedback practical at the individual developer level, not just in dedicated performance engineering teams.

The key discipline is establishing a baseline before optimising. A baseline measurement under a representative workload provides the comparison point against which any change can be evaluated. Without it, improvements cannot be verified and regressions cannot be detected.

6.2 Code Reviews

Code review is a natural point for energy-aware questions that promote shared responsibility without adding rigid gates. Reviewers do not need to be energy measurement specialists; they need a shared vocabulary and a set of prompts that surface potential inefficiencies before they are merged.

Useful questions to introduce into review culture include: - Does this change introduce any new computation on every request, or does it avoid work that previously happened unnecessarily? - Has the data access pattern changed in a way that might increase query volume or payload size? - If a cache is being added, is its size bounded and its hit rate expected to justify the memory cost? - Does this change affect how often background processes run, or how long they hold resources?

These questions do not require profiling data to be useful; they build energy intuition in the team over time and surface issues that measurement can then confirm or rule out.

6.3 Version-Aware Tracking

Linking energy footprint data to specific code versions (commits, branches, or releases) enables trend analysis and regression detection that point-in-time profiling cannot provide. A single measurement tells you the current state; version-aware tracking tells you whether the system is improving, degrading, or stable across the development history.

Prototype systems that tag energy measurements with Git commit identifiers and store them in a queryable database have demonstrated that commit-level energy comparison is practical and developer-friendly. Developers can observe the energy impact of a specific optimisation (e.g., introducing caching), confirm that it persists across subsequent commits, and identify the commit at which a regression was introduced, in the same way they currently track test failures or build times (Joof et al., 2025; Joof, 2025).

This approach treats energy efficiency as a measurable, version-controlled software quality attribute rather than a property that is evaluated informally or not at all.

6.4 CI/CD Integration

Continuous integration and deployment pipelines are the most scalable point for energy feedback, because they run automatically on every change without requiring developer action. Integrating energy profiling into CI pipelines, triggering a

standardised workload execution and collecting energy metrics on each push or pull request, extends the version-aware tracking concept into the automated build process.

Webhook-triggered profiling, where a Git push event automatically deploys the API under test in a containerised environment and executes a predefined workload, has been validated as a practical architecture for this purpose. In a prototype evaluation, the full pipeline from commit to dashboard update completed in under 30 minutes, including container build, workload execution, and data aggregation (Joof et al., 2025).

Automated energy feedback in CI should be designed carefully: - **Informational first:** Surface energy metrics as a report alongside test results, before introducing automated failures for energy regressions. Teams need time to develop intuition for what constitutes a meaningful change before energy gates become useful. - **Selective scope:** Profile the endpoints or components most likely to be affected by a change, rather than running a full workload benchmark on every commit. - **Context-aware thresholds:** Regressions should be flagged relative to a rolling baseline, not a fixed absolute limit, to account for variation in workload and hardware state.

6.5 Visualization and Communication

Raw energy measurements (power readings, CPU percentages, joules per request) are not immediately interpretable by most developers. Effective visualisation translates these numbers into forms that support decisions: trend lines that show whether energy footprint is improving across commits, bar charts that compare design variants side by side, and colour-coded indicators (e.g., red/yellow/green) that communicate regression or improvement at a glance.

Developer feedback from prototype evaluations consistently confirms that visual representations of energy data significantly increase comprehension and willingness to act on the information. A dashboard view that shows how introducing caching reduced both CPU usage and power draw across successive commits “made energy data understandable” in a way that raw log output did not (Joof et al., 2025). Visualisations also support communication with stakeholders who are not directly involved in implementation, making the energy impact of engineering decisions legible to product owners, architects, and sustainability-focused leadership.

Effective energy dashboards include: per-endpoint resource breakdowns, commit-by-commit trend charts, branch comparison views for evaluating design alternatives, and total energy consumed per test run to support longer-term reporting.

6.6 Organizational Adoption

Technical tooling is necessary but not sufficient. Green coding becomes a sustained practice when it has cultural and organisational support: when team leads treat energy as a first-class quality concern, when developers have access to training on what the metrics mean and how to act on them, and when sustainability goals are reflected in how projects are scoped and evaluated.

Research confirms that developers who are personally motivated to reduce energy consumption are often unable to act effectively without institutional support: tools, metrics, management backing, and defined goals (Joof, 2025). Organisations that want to embed green coding practices should therefore invest in all three: tooling that makes energy visible, education that builds intuition, and organisational alignment that treats energy efficiency as a shared engineering responsibility rather than an individual concern.

References

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta–Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyediji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.

7. Trade-offs, Limitations, and Design

Tensions

Green coding involves balancing energy footprint against other software quality attributes. The tensions described here are real, and they cannot be resolved by rule of thumb alone. Each requires empirical evaluation in context.

7.1 CPU vs. Memory

Reducing CPU work often increases memory usage. Precomputing and caching results, for example, avoids repeated calculation at the cost of heap space. The net energy impact depends on the access pattern: memory is relatively cheap to *hold*, but expensive to *allocate*, *copy*, and *garbage-collect* at high rates.

Decision heuristic: Prefer the memory-intensive option when data is accessed repeatedly within a bounded time window and the cache hit rate is demonstrably high. Prefer the compute-intensive option when access is infrequent, data volume makes caching uneconomical, or GC pressure from cached objects outweighs computation savings. Measure both approaches under a representative workload before deciding.

7.2 Network vs. Computation

Reducing data transfer can require additional computation, for example compressing a payload before sending it or pre-aggregating data server-side instead of transferring raw records. Conversely, skipping computation to send data faster keeps the network active longer and may increase device-side processing costs.

Decision heuristic: When user-perceived latency is not the binding constraint, prefer to reduce transfer volume. When latency is critical, benchmark whether the energy cost of additional processing (compression, aggregation) is recovered by reduced transmission time. Note that network energy costs vary substantially between wired, Wi-Fi, and mobile radio contexts. This trade-off has been observed empirically in API-level profiling: adding GZIP compression to API responses reduced network payload but increased CPU load relative to an in-memory caching variant, resulting in higher power consumption (250 mW vs 240 mW) despite smaller transfer sizes, illustrating that compression is not unconditionally beneficial (Joof et al., 2025; Joof, 2025).

7.3 Efficiency vs. Maintainability

Highly optimised code is often harder to understand, test, and change. An energy optimisation that couples components tightly, removes useful abstractions, or relies on undocumented hardware behaviour may reduce energy footprint now while increasing it later through defects and rework.

Decision heuristic: Apply energy-motivated optimisations that do not materially increase code complexity first. When a more invasive optimisation is necessary, document the performance characteristic being preserved, isolate the optimised code behind a well-defined interface, and add benchmarks that will surface regressions if the optimisation is later removed or bypassed.

7.4 Local vs. System-Wide Effects

Optimising a single component in isolation can shift energy consumption elsewhere in the system rather than eliminating it. A faster client-side operation may generate more server requests. A more efficient database query may increase application-layer processing. A reduced network payload may require more CPU to reconstruct on the receiving end.

Decision heuristic: Before optimising a component, trace the data flow through the system to identify downstream effects. Prioritise optimisations that reduce total system energy, not just local metrics. When system-wide measurement is impractical, at minimum verify that the upstream and downstream components are not adversely loaded by the change.

7.5 Measurement Uncertainty

Energy measurement has inherent limitations. Software-accessible counters (such as Intel RAPL) measure processor and DRAM subsystems but exclude storage, network, and peripheral devices. Cloud and virtualised environments often do not expose hardware energy counters at all. Results vary with CPU frequency scaling,

thermal throttling, background load, and measurement granularity.

Decision heuristic: Report relative improvements between a baseline and a candidate under identical conditions, rather than absolute energy values. Repeat measurements across multiple runs and, where possible, across different hardware configurations before drawing conclusions. State measurement conditions explicitly: the tool used, the system state, the workload, and the number of repetitions. Treat a result as preliminary until it has been validated in a representative deployment environment.

7.6 Diminishing Returns

Beyond eliminating major inefficiencies such as redundant computation, unnecessary data transfer, and idle resource holding, additional optimisation yields progressively smaller energy savings at increasing implementation cost. The first 20% of optimisation effort often accounts for 80% of the achievable improvement.

Decision heuristic: Prioritise interventions by estimated energy impact per unit of implementation effort. Use profiling to identify the dominant contributors to energy footprint before investing in optimisation. Stop optimising a component when further improvements require disproportionate complexity or risk, and redirect effort toward higher-impact areas identified by measurement.

References

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta-Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyediji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.

8. Software Energy Footprint

Understanding the software energy footprint requires moving beyond intuition about which code is “fast” or “efficient” and toward a systematic understanding of how software behaviour translates into energy consumption. This section provides the conceptual foundation for that understanding.

8.1 What Is a Software Energy Footprint?

The **software energy footprint** is the total amount of energy consumed by a hardware system as a direct consequence of executing a given piece of software over a defined period or workload. It is not a property of code at rest; it emerges during execution and depends on what the software does, how often, on what hardware, and under what conditions.

This runtime character distinguishes the software energy footprint from lifecycle perspectives that also consider the energy embedded in hardware manufacturing, data centre construction, or end-of-life disposal. Those concerns are real and significant, but they are largely outside the control of a software developer making implementation decisions. The runtime footprint, by contrast, is directly shaped by software design choices and is therefore the primary focus of green coding practice.

The footprint is also inherently **workload-dependent**. The same codebase can have a substantially different energy footprint depending on the size of the input, the number of concurrent users, the frequency of requests, and the presence or absence of data in a cache. This means that energy cannot be meaningfully characterised by inspecting code alone; it requires measurement under representative conditions.

8.2 How Software Consumes Energy

Software does not consume energy directly. It causes hardware components to perform work, and those components consume energy. Understanding which components are involved, and what drives their consumption, is essential for identifying where optimisation effort will have the greatest effect.

Processor (CPU/GPU/TPU) The processor is typically the dominant energy consumer during active computation. Dynamic power, which is the energy

consumed by switching transistors, scales with clock frequency and the number of active circuits. Modern processors use frequency and voltage scaling to reduce power when demand is low, but high utilisation drives them toward their peak power envelope. Software that keeps processors in high-utilisation states for extended periods contributes disproportionately to energy consumption.

Memory Dynamic RAM consumes energy during both access and idle operation. Memory reads and writes, particularly those that miss the CPU cache and reach main memory, are significantly more energy-intensive per operation than operations that remain in the L1 or L2 cache. Software with poor data locality, large working sets, or high allocation and garbage collection rates drives elevated DRAM energy consumption.

Network interfaces Transmitting and receiving data over a network interface requires energy proportional to the volume of data transferred, plus fixed overhead for connection establishment and protocol negotiation. In mobile environments, radio interfaces (LTE, 5G, Wi-Fi) have particularly high energy costs relative to the data transferred, and they remain in an elevated power state for a period after a transmission ends, known as the *radio tail*, which makes many small transmissions significantly more expensive than fewer larger ones (Pathak et al., 2012).

Storage Solid-state storage consumes energy during read and write operations, with writes typically more costly than reads. Spinning disk storage has additional mechanical energy costs. Software that performs unnecessary I/O, such as writing redundant logs, re-reading unchanged files, or performing frequent small random writes, contributes to storage energy consumption in proportion to access frequency and volume.

Display In user-facing applications, the display can be a significant energy consumer, particularly on mobile devices, where screen brightness and rendering complexity directly affect battery drain. Software that keeps the screen active unnecessarily, renders at full resolution when a lower resolution would suffice, or triggers frequent screen updates for non-visible changes increases display energy consumption.

8.3 Attributing Energy to Software

A central challenge in green coding is that energy is measured at the system level as total power drawn from the supply, while responsibility for that consumption is distributed across the operating system, runtime, frameworks, application code, and concurrent processes. Attributing a specific fraction of system energy to a specific software component is technically difficult and subject to significant uncertainty.

Several approaches are used in practice:

Hardware performance counters Modern processors expose energy measurement registers that software can query. Intel's Running Average Power Limit (RAPL) interface, for example, provides per-package and per-domain (CPU, DRAM, uncore) energy readings at high resolution. These are the most direct software-accessible energy measurements available on commodity hardware, but they cover only the processor and memory subsystems and are not available in all virtualised or cloud environments.

System-level power measurement External power meters measure the total energy consumed by a system at the wall or chassis level. This captures all components but does not attribute consumption to individual processes or software components. It is most useful for comparing two versions of a workload running in isolation under identical conditions.

Model-based estimation When direct measurement is unavailable, as is commonly the case in cloud and shared infrastructure, energy consumption can be estimated from CPU utilisation, memory usage, and workload characteristics using empirical models. The accuracy of such estimates varies considerably and depends on the quality of the underlying hardware characterisation. The Cloud Carbon Footprint project and the Software Carbon Intensity (SCI) specification from the Green Software Foundation are examples of frameworks that formalise this approach.

Regardless of the measurement approach, attribution is most reliable when a single workload runs in isolation on known hardware. In production environments, attribution is always partial, and results should be interpreted accordingly (see also [Section 7, Measurement Uncertainty](#)). A practical demonstration of API-level attribution combines containerised execution, which isolates the target workload from background processes, with a software energy profiler such as PowerJoular to attribute power consumption to specific endpoints and commit versions. This approach achieves strong agreement with direct profiler readings (Pearson $r = 0.94$) while requiring only commodity hardware and open-source tooling (Joof et al., 2025; Joof, 2025).

8.4 Energy, Power, and Carbon: Clarifying the Terms

These three related concepts are often conflated, but they are distinct and each has different implications for measurement and optimisation.

Power (measured in watts, W) is the rate of energy consumption at an instant in time. A process that draws 50 W is consuming energy at that rate. Power alone does not tell you the total energy consumed, as that depends on how long the power is drawn.

Energy (measured in joules, J, or kilowatt-hours, kWh) is the product of power and time: $E = P \times t$. A process that draws 50 W for 10 seconds consumes 500 J. Energy is the right unit for comparing the total cost of executing a workload, because it accounts for both the power draw and the duration. A faster but more power-hungry implementation is not necessarily more energy-efficient than a slower, lower-power one, because the total energy (the integral of power over time) determines the footprint.

Carbon emissions (measured in grams or kilograms of CO₂-equivalent, CO₂e) represent the climate impact of energy consumption. The same kilowatt-hour of electricity produces different carbon emissions depending on the source: near-zero for solar or wind, and substantially more for coal or gas. Carbon intensity, measured as grams of CO₂e per kilowatt-hour, varies by grid region, time of day, and season. Optimising for energy footprint and optimising for carbon footprint generally point in the same direction, but the relationship is not fixed, and geographic or temporal shifting of workloads can reduce carbon emissions independently of energy reduction.

For most software optimisation work, **energy** is the most directly actionable metric. Carbon accounting is most relevant at the infrastructure and deployment strategy level, where choices about cloud region, energy procurement, and workload scheduling interact with grid carbon intensity.

8.5 Factors That Shape the Software Energy Footprint

The software energy footprint is not determined by code alone. It is the product of an interaction between software behaviour and execution environment. The principal factors are:

Algorithmic complexity The asymptotic complexity of an algorithm determines how its resource consumption scales with input size. An $O(n^2)$ algorithm applied to a large dataset may consume orders of magnitude more energy than an $O(n \log n)$ alternative. Algorithmic choices therefore have the highest leverage on energy footprint at scale, and they are made early in development, making them among the most important decisions to evaluate from an energy perspective (Pereira et al., 2017). This effect is measurable even at small scale: replacing a custom nested-loop sort with a language built-in sort in a backend API reduced power draw by 23% and CPU utilisation from 34.5% to 26.1% under a standardised request workload (Joof et al., 2025; Joof, 2025).

Data access patterns Whether computations operate on data that is in CPU cache, in main memory, or on disk has a large effect on energy consumption. Cache-friendly data structures and access patterns, particularly those that exhibit spatial and temporal locality, are significantly more energy-efficient than those that scatter memory accesses across large address ranges.

Concurrency and parallelism Parallelism increases hardware utilisation and can reduce wall-clock time, but it does not necessarily reduce energy consumption. If multiple threads perform redundant work, or if synchronisation overhead is high, parallel execution may consume more energy than sequential execution of the same task. Conversely, well-structured parallelism can improve energy efficiency by completing work faster and allowing hardware to return to lower-power states sooner.

Hardware and runtime characteristics The same code produces different energy footprints on different hardware. Processor microarchitecture, memory hierarchy design, and the energy proportionality of the hardware platform (how closely idle power scales with load) all affect the measured footprint. Runtime characteristics such as JIT compilation, garbage collection, and interpreter overhead add additional variability.

Deployment and scaling The number of instances running, the load they serve, and the idle time between requests all affect aggregate energy consumption. A highly optimised single instance running at very low utilisation may consume more energy than a moderately optimised instance running at high utilisation, due to the fixed energy cost of keeping a server running regardless of load.

References

Green Software Foundation (2023) *Software Carbon Intensity (SCI) Specification*. Version 1.0. Available at: <https://sci.greensoftware.foundation>

Pathak, A., Hu, Y.C. and Zhang, M. (2012) *Where is the energy spent inside my app? Fine-grained energy accounting on smartphones with Eprof*. Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys), pp.29–42.

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J.P. and Saraiva, J. (2017) *Energy efficiency across programming languages: How do energy, time, and memory relate?* Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering, pp.256–267.

Treiber, M. (2015) *RAPL in action: Experiences in using RAPL for power measurements*. ACM Transactions on Modeling and Performance Evaluation of Computing Systems, 1(1), pp.1–26.

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta–Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyediji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.

9. Software Sustainability Handprint

Green coding, as described in this guide, is primarily concerned with reducing the direct energy cost of running software. But software also has an indirect environmental dimension: the systems it enables, the behaviours it mediates, and the physical processes it replaces or improves can have positive environmental effects that far exceed the energy cost of the software itself. This positive environmental contribution is referred to as the **software sustainability handprint**.

9.1 From Footprint to Handprint

The concept of a *handprint* originates in sustainability research as a counterpart to the more familiar *footprint*. Where a footprint measures the environmental burden of an activity, including the resources consumed and the emissions generated, a handprint measures the environmental benefit it creates: the footprint that is avoided, reduced, or restored as a result of the activity (Biggs et al., 2020).

Applied to software, the distinction maps cleanly onto two separate questions:

- **Footprint question:** How much energy does this software consume when it runs?
- **Handprint question:** What environmental harm does this software prevent or reduce by functioning as intended?

These are independent. Software with a large footprint can have a large positive handprint, for example a computationally intensive route optimisation system that saves millions of litres of fuel annually. Software with a small footprint can have a negligible handprint, for example a well-optimised but functionally trivial application. And software can, in some cases, have a negative handprint, enabling activities or consumption patterns that increase total environmental harm.

Green coding practice addresses the footprint dimension. Understanding the handprint dimension requires asking a different and broader question: *what does this software make possible, and what is the environmental consequence of that?*

9.2 How Software Creates Positive Environmental Impact

Software creates environmental benefit through several distinct mechanisms. The following categories illustrate the range of impact, from substitution of physical processes to optimisation of physical systems.

Dematerialisation Software can replace physical goods and services that require energy, materials, and logistics to produce and deliver. Digital documents replace printed ones. Video calls replace flights. Streaming media replaces physical discs and the supply chains that produce and distribute them. The environmental gain from dematerialisation is real but not automatic: it depends on whether the digital alternative is actually adopted in place of the physical one, rather than in addition to

it (Hilty and Aebischer, 2015).

Enabling remote work and distributed collaboration Software platforms that support remote work reduce the need for daily commuting and business travel, both of which are significant sources of transport-related emissions. The environmental benefit depends on travel displacement actually occurring, and is offset to a degree by increased residential energy consumption and the energy cost of the collaboration software itself. The net effect is generally positive when long commutes or flights are replaced (Hook et al., 2020).

Optimising physical systems Some of the largest handprint opportunities arise from software that makes physical infrastructure more efficient. Examples include: - *Smart building management systems* that dynamically adjust heating, cooling, and lighting based on occupancy and weather, reducing energy consumption in buildings, which account for a substantial share of global energy use. - *Intelligent transport systems* that optimise traffic flow, reduce idling, and improve public transport scheduling, lowering fuel consumption across vehicle fleets. - *Precision agriculture platforms* that use sensor data and modelling to apply water, fertiliser, and pesticides only where and when they are needed, reducing resource consumption and associated emissions. - *Grid management and demand response software* that integrates renewable energy sources, shifts flexible demand to periods of high renewable generation, and reduces curtailment of wind and solar power.

Enabling scientific and engineering progress Software tools that accelerate research into clean energy, materials science, climate modelling, and low-carbon engineering can have handprint effects that compound over time. The environmental value of a simulation tool that reduces the cost of developing a more efficient battery chemistry, for example, cannot be captured in the software's own energy footprint; it emerges through the downstream applications of the research it enables.

9.3 Evaluating Handprint Claims

The handprint concept is valuable but easily misused. Because handprint effects are indirect and involve counterfactuals (what *would have happened* without the software), they are inherently more difficult to measure and verify than footprint effects. Several principles guide rigorous handprint evaluation.

Additionality A genuine handprint requires that the environmental benefit would not have occurred without the software. If a route optimisation system is deployed in place of a system that was already performing adequate optimisation, the marginal benefit of the new system is the relevant handprint, not the total emissions avoided by any route optimisation whatsoever. Additionality is frequently overstated in sustainability reporting.

Rebound effects Efficiency improvements enabled by software can trigger increases in the activity being made efficient, partially or fully offsetting the environmental gain. More efficient navigation reduces fuel consumption per trip, but cheaper, faster, or more convenient transport can increase the total number of trips. More efficient data compression reduces the energy cost per gigabyte, but lower cost per gigabyte increases total data consumption. These *rebound effects* must be considered when estimating net handprint.

Counterfactual specificity A handprint claim requires specifying what the alternative scenario is. "Software X reduces carbon emissions" is not a complete claim; it must be compared to a defined baseline. Is the alternative no software at all? An earlier version of the software? A manual process? A competitor's product? The magnitude of the handprint depends heavily on which counterfactual is chosen, and the choice should be explicit and justified.

Scope and time horizon Handprint effects can operate over very different timescales. The fuel saved by a route optimisation system is realised immediately, while the effect of accelerating clean-energy research may take decades to materialise. Long time horizons increase uncertainty. Be explicit about the scope of the handprint claim: which effects are included, over what time period, and with what uncertainty.

9.4 Net Impact and the Relationship Between Footprint and Handprint

For most software systems, the direct energy footprint is small relative to the functional value delivered. A well-designed smart thermostat system, for example, might consume a few kilowatt-hours per year in communication and computation while enabling heating and cooling savings of hundreds of kilowatt-hours per year in the buildings it manages. In such cases, the handprint substantially dominates the footprint, and the net environmental impact of the software is clearly positive.

However, this favourable ratio is not universal, and it should not be assumed.

Software that consumes significant energy, such as large-scale data processing, continuous video streaming, or AI model training, can only claim a positive net impact if its handprint effects are real, additional, and large enough to outweigh its footprint. Claims to that effect should be held to the same standard of evidence as any other empirical claim.

For software developers and architects, the practical implication is this: **reducing the footprint and understanding the handprint are complementary, not competing, concerns.** A system with a genuine positive handprint can maximise its net benefit by also minimising its operational footprint. The discipline of thinking rigorously about environmental impact, quantitatively, counterfactually, and with attention to rebound effects, is the same discipline whether it is applied to footprint or handprint.

References

- Biggs, R., de Vos, A., Preiser, R., Clements, H., Maciejewski, K. and Schlüter, M. (eds.) (2020) *The Routledge Handbook of Research Methods for Social-Ecological Systems*. Routledge.
- Hilty, L.M. and Aebischer, B. (2015) *ICT for sustainability: An emerging research field*. In: Hilty, L.M. and Aebischer, B. (eds.) *ICT Innovations for Sustainability*. Springer, pp.3–36.
- Hook, M., Sovacool, B.K. and Sorrell, S. (2020) *A systematic review of the energy and climate impacts of teleworking*. *Environmental Research Letters*, 15(9), 093003.
- Plepyś, A. (2002) *The grey side of ICT*. *Environmental Impact Assessment Review*, 22(5), pp.509–523.

10. Green Coding Checklists

These checklists support reflection and discussion during code review, design review, or retrospectives, not as compliance audits. Use them as prompts to identify energy-related risks, not as a pass/fail gate.

10.1 Frontend – Web

- Are re-renders limited to the components that actually changed?
- Has bundle size been measured recently, and is it justified?
- Is data fetched only when needed, and not fetched more than once for the same content?
- Are images and media served at appropriate resolution and in modern formats (WebP, AVIF)?
- Are third-party scripts audited for necessity and load impact?
- Are CSS transitions or the Web Animations API preferred over JavaScript animation loops?
- Are polling or interval-based patterns replaced with event-driven or push alternatives where feasible?
- Is HTTP caching configured correctly for static and semi-static assets?

10.2 Frontend – Mobile

- Are background tasks minimized, consolidated, and deferred to system-scheduled windows?
- Are sensors and wakeup receivers unregistered when not actively needed?
- Is offline behavior supported to avoid redundant network round trips?
- Are network requests batched so the radio can power down between active periods?
- Is rendering limited to visible content using view recycling or lazy list patterns?
- Is location accuracy matched to actual requirements (coarse vs. fine)?
- Are push notifications used in preference to polling for state updates?
- Are animations and transitions implemented using hardware-accelerated APIs?

10.3 Backend – Services

- Is work per request profiled, and are hot paths free of avoidable operations?
- Are database queries selective, indexed, and paginated rather than fetching full tables?
- Are N+1 query patterns eliminated through eager loading or batched queries?
- Is payload size minimized, and are only required fields returned?
- Are caches bounded, and are hit rates measured to confirm they are earning

their memory cost?

- Are synchronous operations that could be safely deferred moved off the critical path?
- Are idle connections, thread pools, and executors bounded to avoid resource exhaustion under low load?
- Is serialization format appropriate to the use case (e.g., binary formats for internal high-throughput paths)?

10.4 Data Pipelines

- Is recomputation of unchanged partitions avoided through checksums, timestamps, or incremental frameworks?
- Are checkpoints used so that failed runs can resume rather than restart from scratch?
- Is processing frequency matched to actual downstream consumption rate?
- Is cold or infrequently accessed data tiered to lower-energy storage?
- Is data retention governed by a defined lifecycle policy that prevents unbounded growth?
- Are pipeline stages profiled to identify which step dominates energy and execution time?
- Is parallelism matched to available hardware, avoiding both under- and over-provisioning?

10.5 AI Training

- Is model capacity justified by the task, or would a smaller architecture meet requirements?
- Is experiment tracking in place to prevent duplicate or near-duplicate training runs?
- Is GPU/TPU utilization consistently high, or are there data-loading or preprocessing bottlenecks?
- Is early stopping applied when validation metrics plateau?
- Is mixed-precision training (FP16/BF16) used where hardware and framework support it?
- Are datasets filtered or deduplicated to remove samples that do not contribute to model quality?
- Are hyperparameter searches bounded and stopped when diminishing returns are evident?

10.6 AI Inference

- Is the smallest model that meets quality requirements used in production?
- Are inputs and prompts as concise as possible, removing redundant context?
- Is caching applied to deterministic or repeated queries to avoid redundant inference?
- Are requests batched where latency requirements allow?
- Has model quantization been evaluated as a way to reduce per-request compute?
- Is inference endpoint utilization monitored, and is capacity adjusted to avoid sustained idle over-provisioning?
- Are embedding computations cached rather than recomputed on each request?

11. Outlook and Future Directions

Green coding is an evolving discipline. The principles and practices described in this guide represent the current state of evidence-based understanding, but several important gaps remain, in tooling, education, organisational practice, and research, that will shape how the field develops.

Better measurement tooling. Most existing energy profilers are designed for system-level or process-level analysis and lack integration with the tools developers use daily. Lightweight, workflow-integrated profiling that connects energy measurements to version control commits, containerised test environments, and visual dashboards is a direction actively under development. Prototype research has demonstrated the technical feasibility of commit-aware, reproducible API energy profiling on accessible hardware, with future work extending toward cloud-based CI/CD pipelines, multi-profiler orchestration, and carbon-intensity mapping (Joof et al., 2025; Joof, 2025).

Education and skill development. Developers are often unaware of how their design and implementation decisions affect energy consumption, not because they lack interest, but because energy has not been part of standard software engineering education or tooling feedback loops. Embedding energy awareness into curricula, code review practices, and IDE tooling is necessary for green coding to move from specialist practice to engineering norm.

Organisational alignment. Technical tools are only effective when organisations treat energy efficiency as a first-class quality concern, alongside performance, reliability, and maintainability. This requires defined sustainability goals, supported by metrics and integrated into project planning, not treated as optional or post-hoc.

Research-industry collaboration. The most significant open questions in green coding, including how to measure energy attributably in distributed and cloud-native systems, how to reason about system-wide trade-offs, and how to validate the long-term effectiveness of green coding practices at scale, require sustained collaboration between researchers and practitioners. Initiatives that ground research in real development environments and produce artefacts that practitioners can adopt are essential to closing the gap between academic findings and applied practice.

By treating energy footprint as a first-class software quality attribute, green coding enables better decisions, not perfection, but continuous improvement grounded in evidence. The tools, practices, and organisational conditions needed to sustain that improvement are within reach, and this guide is intended as a contribution toward them.

References

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta–Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyediji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.